

Nama: Hans Malvin Djojo

<https://github.com/hansmalvin/LastMile>

Environment Setup

mongosh

use salesData

db.createCollection("sales")

```
db.sales.insertMany([
  {
    "saleDate": ISODate("2023-01-15T00:00:00Z"),
    "customer": { "customerId": "C1001", "name": "Alice Smith" },
    "product": { "productId": "P201", "name": "Laptop", "category": "Electronics" },
    "amount": 1200,
    "quantity": 1
  },
  {
    "saleDate": ISODate("2023-02-20T00:00:00Z"),
    "customer": { "customerId": "C1002", "name": "Bob Johnson" },
    "product": { "productId": "P202", "name": "Tablet", "category": "Electronics" },
    "amount": 300,
    "quantity": 2
  },
  {
    "saleDate": ISODate("2023-03-10T00:00:00Z"),
    "customer": { "customerId": "C1001", "name": "Alice Smith" },
    "product": { "productId": "P301", "name": "T-shirt", "category": "Clothing" },
    "amount": 25,
    "quantity": 5
  },
  {
    "saleDate": ISODate("2023-04-05T00:00:00Z"),
    "customer": { "customerId": "C1003", "name": "Charlie Brown" },
    "product": { "productId": "P201", "name": "Laptop", "category": "Electronics" },
    "amount": 1100,
    "quantity": 1
  },
  {
    "saleDate": ISODate("2023-05-12T00:00:00Z"),
    "customer": { "customerId": "C1002", "name": "Bob Johnson" },
    "product": { "productId": "P302", "name": "Jeans", "category": "Clothing" },
    "amount": 60,
    "quantity": 1
  },
  {
```

```
"saleDate": ISODate("2023-06-22T00:00:00Z"),
"customer": { "customerId": "C1004", "name": "David Lee" },
"product": { "productId": "P401", "name": "Coffee Maker", "category": "Appliances" },
"amount": 80,
"quantity": 1
},
{
"saleDate": ISODate("2023-07-01T00:00:00Z"),
"customer": { "customerId": "C1001", "name": "Alice Smith" },
"product": { "productId": "P202", "name": "Tablet", "category": "Electronics" },
"amount": 320,
"quantity": 2
},
{
"saleDate": ISODate("2023-08-18T00:00:00Z"),
"customer": { "customerId": "C1003", "name": "Charlie Brown" },
"product": { "productId": "P301", "name": "T-shirt", "category": "Clothing" },
"amount": 30,
"quantity": 6
},
{
"saleDate": ISODate("2023-09-05T00:00:00Z"),
"customer": { "customerId": "C1005", "name": "Eve Wilson" },
"product": { "productId": "P201", "name": "Laptop", "category": "Electronics" },
"amount": 1250,
"quantity": 1
},
{
"saleDate": ISODate("2023-10-11T00:00:00Z"),
"customer": { "customerId": "C1004", "name": "David Lee" },
"product": { "productId": "P302", "name": "Jeans", "category": "Clothing" },
"amount": 70,
"quantity": 1
}
])
```

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69fddc80fe23c058ad3682d1'),
    '1': ObjectId('69fddc80fe23c058ad3682d2'),
    '2': ObjectId('69fddc80fe23c058ad3682d3'),
    '3': ObjectId('69fddc80fe23c058ad3682d4'),
    '4': ObjectId('69fddc80fe23c058ad3682d5'),
    '5': ObjectId('69fddc80fe23c058ad3682d6'),
    '6': ObjectId('69fddc80fe23c058ad3682d7'),
    '7': ObjectId('69fddc80fe23c058ad3682d8'),
    '8': ObjectId('69fddc80fe23c058ad3682d9'),
    '9': ObjectId('69fddc80fe23c058ad3682da')
  }
}
salesData>
```

db.sales.find().pretty()

```

salesData> db.sales.find().pretty()
[
  {
    _id: ObjectId('69fddc80fe23c058ad3682d1'),
    saleDate: ISODate('2023-01-15T00:00:00.000Z'),
    customer: { customerId: 'C1001', name: 'Alice Smith' },
    product: { productId: 'P201', name: 'Laptop', category: 'Electronics' },
    amount: 1200,
    quantity: 1
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d2'),
    saleDate: ISODate('2023-02-20T00:00:00.000Z'),
    customer: { customerId: 'C1002', name: 'Bob Johnson' },
    product: { productId: 'P202', name: 'Tablet', category: 'Electronics' },
    amount: 300,
    quantity: 2
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d3'),
    saleDate: ISODate('2023-03-10T00:00:00.000Z'),
    customer: { customerId: 'C1001', name: 'Alice Smith' },
    product: { productId: 'P301', name: 'T-shirt', category: 'Clothing' },
    amount: 25,
    quantity: 5
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d4'),
    saleDate: ISODate('2023-04-05T00:00:00.000Z'),
    customer: { customerId: 'C1003', name: 'Charlie Brown' },
    product: { productId: 'P201', name: 'Laptop', category: 'Electronics' },
    amount: 1100,
    quantity: 1
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d5'),
    saleDate: ISODate('2023-05-12T00:00:00.000Z'),
    customer: { customerId: 'C1002', name: 'Bob Johnson' },
    product: { productId: 'P302', name: 'Jeans', category: 'Clothing' },
    amount: 60,
    quantity: 1
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d6'),
    saleDate: ISODate('2023-06-22T00:00:00.000Z'),
    customer: { customerId: 'C1004', name: 'David Lee' },
    product: { productId: 'P401', name: 'Coffee Maker', category: 'Appliances' },
    amount: 80,
    quantity: 1
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d7'),
    saleDate: ISODate('2023-07-01T00:00:00.000Z'),
    customer: { customerId: 'C1001', name: 'Alice Smith' },
    product: { productId: 'P202', name: 'Tablet', category: 'Electronics' },
    amount: 320,
    quantity: 2
  },
  {
    _id: ObjectId('69fddc80fe23c058ad3682d8'),
    saleDate: ISODate('2023-08-18T00:00:00.000Z'),
    customer: { customerId: 'C1003', name: 'Charlie Brown' },
    product: { productId: 'P301', name: 'T-shirt', category: 'Clothing' },
    amount: 30,
    quantity: 6
  }
]

```

```

{
  _id: ObjectId('69fddc80fe23c058ad3682d4'),
  saleDate: ISODate('2023-04-05T00:00:00.000Z'),
  customer: { customerId: 'C1003', name: 'Charlie Brown' },
  product: { productId: 'P201', name: 'Laptop', category: 'Electronics' },
  amount: 1100,
  quantity: 1
},
{
  _id: ObjectId('69fddc80fe23c058ad3682d5'),
  saleDate: ISODate('2023-05-12T00:00:00.000Z'),
  customer: { customerId: 'C1002', name: 'Bob Johnson' },
  product: { productId: 'P302', name: 'Jeans', category: 'Clothing' },
  amount: 60,
  quantity: 1
},
{
  _id: ObjectId('69fddc80fe23c058ad3682d6'),
  saleDate: ISODate('2023-06-22T00:00:00.000Z'),
  customer: { customerId: 'C1004', name: 'David Lee' },
  product: { productId: 'P401', name: 'Coffee Maker', category: 'Appliances' },
  amount: 80,
  quantity: 1
},
{
  _id: ObjectId('69fddc80fe23c058ad3682d7'),
  saleDate: ISODate('2023-07-01T00:00:00.000Z'),
  customer: { customerId: 'C1001', name: 'Alice Smith' },
  product: { productId: 'P202', name: 'Tablet', category: 'Electronics' },
  amount: 320,
  quantity: 2
},
{
  _id: ObjectId('69fddc80fe23c058ad3682d8'),
  saleDate: ISODate('2023-08-18T00:00:00.000Z'),
  customer: { customerId: 'C1003', name: 'Charlie Brown' },
  product: { productId: 'P301', name: 'T-shirt', category: 'Clothing' },
  amount: 30,
  quantity: 6
},
{
  _id: ObjectId('69fddc80fe23c058ad3682d9'),
  saleDate: ISODate('2023-09-05T00:00:00.000Z'),
  customer: { customerId: 'C1005', name: 'Eve Wilson' },
  product: { productId: 'P201', name: 'Laptop', category: 'Electronics' },
  amount: 1250,
  quantity: 1
},
{
  _id: ObjectId('69fddc80fe23c058ad3682da'),
  saleDate: ISODate('2023-10-11T00:00:00.000Z'),
  customer: { customerId: 'C1004', name: 'David Lee' },
  product: { productId: 'P302', name: 'Jeans', category: 'Clothing' },
  amount: 70,
  quantity: 1
}
]
salesData>

```

Step 1: Calculate Total Sales per Customer

```

db.sales.aggregate([
{
  $group: {
    _id: "$customer.customerId",
    totalSales: { $sum: "$amount" },
    customerName: { $first: "$customer.name" } // Add customer name for clarity
  },
{
  $project: {
    _id: 0, // Exclude the default _id field
    customerId: "$_id",
    customerName: 1,

```

```

totalSales: 1
}
},
{
$sort: { totalSales: -1 } // Sort by total sales in descending order
}
])

```

Explanations:

To calculate the total sales per customer, the pipeline first organizes the raw data into distinct buckets for every unique customer ID using the `$group` stage. Within these buckets, the database performs two simultaneous tasks: it calculates the total spending by summing up the amount field and captures the customer's name to ensure the results are identifiable. Following this consolidation, a `$project` stage is used to refine the output by hiding the internal database IDs and assigning more readable labels to the fields, while a final `$sort` stage arranges the entries in descending order to place the highest-spending customers at the top of the leaderboard.

```

salesData> db.sales.aggregate([
| {
|   $group: {
|     _id: "$customer.customerId",
|     totalSales: { $sum: "$amount" },
|     customerName: { $first: "$customer.name" } // Add customer name for clarity
|   },
|   $project: {
|     _id: 0, // Exclude the default _id field
|     customerId: "$_id",
|     customerName: 1,
|     totalSales: 1
|   },
|   $sort: { totalSales: -1 } // Sort by total sales in descending order
| })
|
| [
|   {
|     totalSales: 1545, customerName: 'Alice Smith', customerId: 'C1001' },
|     totalSales: 1250, customerName: 'Eve Wilson', customerId: 'C1005' },
|     totalSales: 1130,
|     customerName: 'Charlie Brown',
|     customerId: 'C1003'
|   },
|   {
|     totalSales: 360, customerName: 'Bob Johnson', customerId: 'C1002' },
|     totalSales: 150, customerName: 'David Lee', customerId: 'C1004' }
| ]
salesData>

```

Step 2: Find the Most Popular Product Category

```

db.sales.aggregate([
{
$group: {

```

```

    _id: "$product.category",
    totalQuantity: { $sum: "$quantity" }
  },
  {
    $sort: { totalQuantity: -1 }
  },
  {
    $limit: 1 // Return only the top category
  },
  {
    $project: {
      _id: 0,
      category: "$_id",
      totalQuantity: 1
    }
  }
])

```

Explanations:

Determining the most popular product category involves a similar process of grouping data, though the focus shifts to item categories rather than individual people. The `$group` stage consolidates the data by category type and calculates the total quantity of items sold for each group. To identify the "best seller," the pipeline then sorts these categories from highest to lowest volume and uses a `$limit` stage to keep only the single top result. Finally, the `$project` stage cleans up the result by renaming the grouping identifier to a clearer "category" label and displaying the total count.

```

salesData> db.sales.aggregate([
| {
|   $group: {
|     _id: "$product.category",
|     totalQuantity: { $sum: "$quantity" }
|   },
|   $sort: { totalQuantity: -1 }
| },
| {
|   $limit: 1 // Return only the top category
| },
| {
|   $project: {
|     _id: 0,
|     category: "$_id",
|     totalQuantity: 1
|   }
| }
| ])
|
| [ { totalQuantity: 13, category: 'Clothing' } ]
salesData>

```

Step 3: Analyze Sales by Month

```

db.sales.aggregate([
| {
|   $group: {
|     _id: { $month: "$saleDate" },
|     totalSales: { $sum: "$amount" }
|   },
| },
| {
|   $project: {
|     _id: 0,
|     month: "$_id",
|     totalSales: 1
|   }
| },
| {
|   $sort: { month: 1 } // Sort by month in ascending order
| }
| ])

```

Explanations:

Analyzing sales trends by month requires extracting temporal data from the sale dates using the \$month operator within a \$group stage. This operation ignores the specific day or year of a transaction, effectively merging all sales from January together, February together, and

so on, to calculate a total sales figure for each month. The \$project stage then reshapes the data by mapping these monthly identifiers to a standard "month" field, and a \$sort stage is applied to ensure the final report flows chronologically from the start of the year to the end rather than by sales volume.

```
salesData>
| db.sales.aggregate([
|   {
|     $group: {
|       _id: { $month: "$saleDate" },
|       totalSales: { $sum: "$amount" }
|     },
|   },
|   {
|     $project: {
|       _id: 0,
|       month: "$_id",
|       totalSales: 1
|     },
|   },
|   {
|     $sort: { month: 1 } // Sort by month in ascending order
|   }
| ])
|
| [
|   { totalSales: 1200, month: 1 },
|   { totalSales: 300, month: 2 },
|   { totalSales: 25, month: 3 },
|   { totalSales: 1100, month: 4 },
|   { totalSales: 60, month: 5 },
|   { totalSales: 80, month: 6 },
|   { totalSales: 320, month: 7 },
|   { totalSales: 30, month: 8 },
|   { totalSales: 1250, month: 9 },
|   { totalSales: 70, month: 10 }
| ]
salesData> |
```

Step 4: Find Customers Who Purchased Laptops

```
db.sales.aggregate([
{
$match: { "product.name": "Laptop" }
},
{
$group: {
_id: "$customer.customerId",
customerName: { $first: "$customer.name" }
}
},
{

```

```

$project: {
  _id: 0,
  customerId: "$_id",
  customerName: 1
}
}
])

```

Explanations:

To identify specific buyer behaviors, such as who purchased laptops, the pipeline begins with a targeted `$match` stage that acts as an initial filter. This stage discards any transaction that does not involve a laptop before any further processing occurs, which significantly improves efficiency by reducing the volume of data passed to the next step. The remaining records are then funneled into a `$group` stage to deduplicate the results, ensuring that each customer is only listed once regardless of how many units they bought, before the `$project` stage formats the final list of names and IDs.

```

salesData> db.sales.aggregate([
| {
|   $match: { "product.name": "Laptop" }
| },
| {
|   $group: {
|     _id: "$customer.customerId",
|     customerName: { $first: "$customer.name" }
|   }
| },
| {
|   $project: {
|     _id: 0,
|     customerId: "$_id",
|     customerName: 1
|   }
| }
| ])
|
| [
|   { customerName: 'Charlie Brown', customerId: 'C1003' },
|   { customerName: 'Alice Smith', customerId: 'C1001' },
|   { customerName: 'Eve Wilson', customerId: 'C1005' }
| ]
salesData>

```

Step 5: Calculate Average Purchase Amount

```

db.sales.aggregate([
{
  $group: {
    _id: null, // Group all documents into a single group
    averageAmount: { $avg: "$amount" }
  }
}

```

```
},  
{  
  $project: {  
    _id: 0,  
    averageAmount: 1  
  }  
}  
])
```

Explanations:

Calculating a single benchmark like the average purchase amount involves treating the entire database as a single group rather than splitting it into categories. By setting the identifier to null in the \$group stage, you command MongoDB to put every single record into one giant bucket. Once combined, the \$avg operator computes the mathematical mean of every amount field in the collection, and a final \$project stage removes the null identifier to present only the final average value as a clean, standalone result.

```
salesData> db.sales.aggregate([  
  {  
    $group: {  
      _id: null, // Group all documents into a single group  
      averageAmount: { $avg: "$amount" }  
    },  
  },  
  {  
    $project: {  
      _id: 0,  
      averageAmount: 1  
    }  
  }  
])  
  
[ { averageAmount: 443.5 } ]  
salesData>
```